AI-Driven Quality Assistance Application for STLC Automation

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE DEGREE OF

Master of Technology (Online)
IN
Data Sciences and Business Analytics

BY

Monika Tyagi



Faculty of Engineering
Indian Institute of Science
Bangalore – 560 012 (INDIA)

November, 2026

Declaration of Originality

I, **Monika Tyagi**, with SR No. **13-19-02-19-52-24-1-24515** hereby declare that the material presented in the project report titled

AI-Driven Quality Assistance Application for STLC Automation

represents original work carried out by me at **Alight Services India Pvt Ltd** as part of the project credit requirements of the **Master of Technology (Online)** degree in **Data Science and Business Analytics** at the Indian Institute of Science, between **August, 2024** to **November, 2026**.
With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Date: Student Signature

In our capacities as internal project guide and faculty mentor of the above-mentioned work, we certify that the above statements are true to the best of our knowledge, and we have carried out due diligence to ensure the originality of the report.

Internal Guide Name: *Kapil Madaan* Internal Guide Signature
Organisation: *Alight Services India Pvt Ltd*

Faculty Mentor Name: *Punit Rathore* Faculty Mentor Signature
Department: *Cyber Physical Systems*

© Monika Tyagi

November, 2026

All rights reserved

Acknowledgements

I would like to sincerely thank Prof. Punit Rathore, my faculty mentor at IISc, for his constant guidance and support throughout this project. His suggestions helped me understand how to approach the problem and make the right decisions at different stages. Whenever I faced difficulties during implementation, he took the time to guide me and suggest possible ways forward. His inputs played an important role in shaping the overall design and direction of this project.

I am also thankful to Prof. Deepak Subramani for guiding me through the overall project process and report preparation. His advice on organising the work, deciding what to focus on during evaluation, and presenting the results helped me stay on track. His guidance made the process of completing the project and documenting the work much easier.

I would also like to thank Mr. Kapil Madaan, my company mentor, for his support during the design and development of the system. His practical suggestions helped me think through different technical choices and make better decisions about the architecture. His guidance was particularly helpful whenever I encountered challenges during implementation.

Lastly, I am grateful to my family and friends for their understanding, patience, and encouragement throughout the M.Tech. programme. Their support gave me the motivation to keep going, especially during the more demanding phases of this journey.

Abstract

The Software Testing Life Cycle (STLC) involves several activities, starting with understanding requirements and planning tests, followed by test design, execution, defect reporting, and release decisions. In most development teams, these activities are handled using different tools, often requiring manual effort to move information from one stage to another. This project explores how AI can help bring these activities together through a central orchestrator supported by a group of specialised agents. The aim is to help testing teams make better use of requirements, user stories, code changes, and test execution results to generate useful quality-related insights, while keeping important decisions, such as reviewing results and approving releases, under human control.

The prototype developed as part of this project is a web-based application built using React and TypeScript for the frontend, FastAPI for the backend, and PostgreSQL for data storage. A separate agent service, developed using Google's Agent Development Kit (ADK), handles AI-related tasks. The backend manages user authentication, stores assistance requests, and communicates with the agent service. At the centre of the system is a root agent that identifies the nature of a request and directs it to one of nine specialised agents. These agents focus on different STLC activities, including test-case generation, test-data generation, test automation suggestions, test execution analysis, self-healing recommendations, change-impact analysis, defect reporting, test-summary preparation, and release-readiness assessment. The application also includes integrations for importing Jira Cloud stories and retrieving code changes from GitHub. It supports optional Microsoft Entra single sign-on and allows different model providers to be configured.

This report covers the motivation behind the project, relevant research on software testing

and AI agents, the proposed architecture, implementation details, and security considerations. It also discusses how the system can be evaluated in a practical setting. An important distinction is made between the features currently implemented in the prototype and the capabilities proposed for future development. For example, the test execution analysis agent works with available test evidence but does not execute external test suites. Similarly, self-healing recommendations are provided as suggestions rather than being applied automatically. No completed comparative benchmark or measured performance improvement is claimed in this report.

The prototype brings several testing-related activities into a single application and demonstrates how specialised AI agents can support different stages of the STLC. However, its actual effectiveness, reliability, and suitability for use in real-world projects will need to be assessed through further testing and controlled evaluation. Future evaluation can focus on the quality of generated outputs, traceability, time and effort saved, execution efficiency, and the accuracy of defect analysis, with human review remaining an important part of the process.

Contents

Acknowledgements i

Abstract ii

Contents iv

List of Figures vii

List of Tables viii

List of Algorithms ix

Acronyms, Abbreviations and Symbols xi

1 Introduction 1

1.1 Background 1

1.2 Problem Statement and Motivation 2

1.3 Research Questions 2

1.4 Objectives 3

1.5 Scope and Contributions 3

1.6 Report Organization 4

2 Literature Review 5

2.1 Software Testing Life Cycle 5

2.2 LLMs in Software Quality Engineering 6

2.3 Agent-Orchestrated Workflows . . . 6
2.4 Grounding, Traceability, and Human Review . . . 7
2.5 Evaluation of AI-Assisted Testing . . . 7

3 Methodology and Implementation 8
3.1 Development Method . . . 8
3.2 Requirements and Actors . . . 9
3.3 Technology Stack . . . 9
3.4 System Architecture . . . 9
3.4.1 Frontend . . . 11
3.4.2 Backend and Persistence . . . 11
3.4.3 Agent Orchestration . . . 12
3.4.4 External Integrations and Model Providers . . . 14
3.5 Security and Data Handling . . . 15
3.6 Deployment and Operations . . . 15
3.7 Quality Attributes . . . 15

4 Results and Discussions 17
4.1 Prototype Implementation Status . . . 17
4.2 Existing Automated Test Assets . . . 17
4.3 Demonstration Scenarios . . . 17
4.3.1 Interface Walkthrough and Illustrative Outputs . . . 19
4.4 Evaluation Plan . . . 23
4.4.1 Dataset and Protocol . . . 23
4.4.2 Measures . . . 24
4.4.3 Analysis and Validity . . . 24
4.5 Results and Discussion . . . 25
4.6 Limitations . . . 25

5 Conclusions **27**

5.1 Future Work 28

Appendix **29**

5.A Representative Request and Response Flow 29

5.A Reproducibility Checklist for a Future Evaluation 29

5.A Project Materials 30

References **31**

List of Figures

3.1 Logical application architecture. Optional external integrations and model providers are enabled by configuration. 10
3.2 High-level design of the Quality Assistance application. 11
3.3 Illustrative agentic workflow showing orchestration among the root agent and specialist agents. 13
3.4 Illustrative request sequence through the application services. 14

4.1 Workspace dashboard used to start a quality-assistance request. 19
4.2 Settings page for configuring Jira, GitHub, and sign-in integrations. Availability depends on administrator configuration and valid credentials. 20
4.3 Example request input containing quality context and a user story before agent processing. 20
4.4 Example generated response with prioritized test-case recommendations. 21
4.5 Example suggested test data presented as part of the generated response. 21
4.6 Example automation notes and test-script recommendations. 22
4.7 Example release-risk and readiness analysis based on the submitted context. . . 23

List of Tables

3.1 Selected functional requirements and corresponding implemented behavior. . . . 9
3.2 Technology stack used in the prototype. 10
3.3 Registered specialist agents and the scope described by their instructions. 12

4.1 Implementation status as of November 2026, distinguishing current behavior from planned work. 18
4.2 Proposed evaluation measures; no values are reported because a controlled evaluation has not yet been completed. 24

List of Algorithms

No explicit algorithms beyond standard library implementations are used in this project. The core inference logic is delegated entirely to the Google ADK agents and the configured LLM.

Acronyms, Abbreviations and Symbols

These short forms are used in more than one chapter.

Acronyms

ADK Agent Development Kit

API Application Programming Interface

CI/CD Continuous Integration and Continuous Delivery

LLM Large Language Model

M.Tech. Master of Technology

MCP Model Context Protocol

QA Quality Assurance

RQ Research Question

STLC Software Testing Life Cycle

UI User Interface

Abbreviations

E2E End-to-end

JWT JSON Web Token

ACRONYMS, ABBREVIATIONS AND SYMBOLS

RAG Retrieval-Augmented Generation

REST Representational State Transfer

Chapter 1

Introduction

1.1 Background

Running tests is only one part of software quality. In the Software Testing Life Cycle (STLC) the team first reads the requirement, decides the scope, designs cases and data, executes, logs defects, writes a summary, and then, often under time pressure, says whether a release can go. ISO/IEC/IEEE 29119 and the ISTQB Foundation syllabus both treat this as planned work that should stay traceable to risk [1, 2].

The part that slowed people down, in my experience, was not any single step. It was the spread across tools. A story arrives as a ticket. A change arrives as a code difference. A failure arrives as a log. Somebody still has to turn the story into tests, decide what the code difference should regress, and turn the run into something a release manager can read. On one ticket this is ordinary. Across a sprint it is the same interpretation done again and again, and the wording drifts.

Large language models (LLMs) handle the language part of that reasonably well. They can summarise a story, suggest boundary cases, or drop a messy log into a defect template. They are weaker when no one is watching. I have seen models skip a rule that was in the second paragraph, add a field that was never there, and still sound certain. For a quality team that is a problem. An assistant is useful only if it takes the context the user gives it, stays on the task, shows the draft, and waits for a person to accept or throw it away.

1.2 Problem Statement and Motivation

The main motivation behind this project is the need for a more connected and intelligent way of managing different activities within the Software Testing Life Cycle (STLC). In many teams, quality-related tasks are handled separately, and information often needs to be reviewed and interpreted again at each stage. Although automation has made individual testing activities easier, tasks such as understanding requirements, identifying potential defects, and assessing release readiness still involve considerable manual effort. This lack of coordination can result in important test scenarios being missed, work being repeated, and potential risks being identified late in the development process.

To address these challenges, this project proposes an AI-assisted system that brings different STLC activities together through a central orchestrator. Users can provide requirements and user stories, along with source-code changes or test execution evidence, depending on the task. A root agent analyses the request and assigns the relevant work to specialised agents. Their outputs are then combined into a single response to help the quality team understand what needs attention and take appropriate action.

The system is intended to support engineers rather than replace their judgement. It helps prepare and organise testing-related information and provides suggestions that can be reviewed before being used. The responsibility for validating test cases, running tests in the appropriate environment, confirming defects, and making final release decisions remains with the concerned engineers.

1.3 Research Questions

The project is guided by the following research questions:

1. **RQ1:** How can a multi-agent AI system coordinate several STLC activities through one quality-assistance interface?

2. **RQ2:** What validation, provenance, and human-review controls are needed before AI-generated testing artifacts can be trusted enough to use?

3. **RQ3:** How should the system be evaluated for its effect on requirement-to-test traceability, quality-engineering effort, coverage, and defect analysis?

1.4 Objectives

The main objectives of this project are to:

- design and develop a full-stack application where users can submit software-testing information and review the results generated with AI assistance;
- build a central agent that can understand user requests and delegate tasks to specialised agents covering different activities of the STLC;
- provide secure, authenticated access to the application and maintain user-specific records of submitted requests and their results;
- integrate with engineering tools such as Jira for accessing user stories and GitHub for retrieving source-code changes;
- allow users to configure different AI model providers and application deployment settings; and
- establish a practical and reproducible evaluation approach in which the generated results are reviewed by humans, without making claims about improvements that have not yet been measured.

1.5 Scope and Contributions

This report is about the code in the project repository [3] as of November 2026. What I am submitting is a working prototype. I am not offering a new theory of agents. The browser UI, the API, and the agent service run together. Users can paste requirements and stories, attach git code differences or a link to a GitHub pull request with the changes, add a test log, continue in a later turn, and export the reply. If Jira Cloud and GitHub are set up, issue text and

pull-request or branch code differences can be fetched instead of typed. The root agent then works with nine STLC specialists.

There are limits I do not want to hide. The prototype will not run someone else's suite. It will not rewrite scripts in a git repository. It cannot prove that the generated cases are complete, and it cannot approve a release. Execution, self-healing, and readiness answers are suggestions from the text that was submitted. I also did not implement retrieval over a test repository, CI/CD execution, strict output schemas, or a comparative study. Those are listed later as future work.

1.6 Report Organization

Chapter 2 is the reading I used: STLC, LLMs in testing, agent setups, and the validation issues. Chapter 3 is how I built the prototype, including architecture and security. Chapter 4 is what I can actually show, plus the evaluation I still need for the research questions. Chapter 5 is the wrap-up and the list of next steps.

Chapter 2

Literature Review

2.1 Software Testing Life Cycle

Software testing is an important part of software development. It involves a series of planned activities to check whether a software product works as expected and to identify areas where improvements may be needed. The ISO/IEC/IEEE 29119 series provides guidelines for software testing, including its processes, concepts, and documentation. Similarly, the ISTQB Foundation syllabus explains the key activities involved in testing, such as planning, analysing requirements, designing test cases, preparing tests, executing them, and completing the testing process [1, 2]. Both approaches emphasise the importance of maintaining traceability throughout testing. This means that test cases and their results should be linked to the original requirements and identified risks, so that quality-related decisions can be supported by actual evidence.

In practice, the Software Testing Life Cycle (STLC) does not always follow a fixed sequence of steps. The outcome of one activity can influence what happens next. For example, testing may reveal gaps or unclear points in the requirements, while identifying a defect may require additional regression test cases. Similarly, changes in the source code can affect which parts of the application need to be tested again. Because of these dependencies, an effective system should be able to retain information from earlier stages and use it when supporting later activities. Rather than generating separate outputs for each testing task, an AI-assisted system should help maintain the connection between different stages and make it easier for engineers

to understand how one activity relates to another.

2.2 LLMs in Software Quality Engineering

LLMs can read a story and write scenarios. They can suggest data, classify a failure, or summarise a run. That is why people reach for them: the jobs are repetitive and mostly text. Reliability is a different matter. A model may ignore a constraint, invent a step, or write as if it is sure when the input is thin.

Amershi et al. make a similar point for machine learning in software teams: the workflow has to stay with people, ownership has to be clear, data quality has to be watched, and the system has to be monitored after it ships [4]. I treated that as something I had to design for. In this assistant the model output is a draft. A tester should be able to check the assumptions, edit the cases, and drop a recommendation that is not supported, before it goes into a plan or a release discussion.

2.3 Agent-Orchestrated Workflows

The usual agent pattern is a set of specialists and a coordinator that decides who to call. ReAct is one way of mixing reasoning with actions. Broader surveys talk about planning, tools, memory, and how agents are coordinated [5, 6]. Splitting the work made prompts easier for me to maintain. It did not make the answers correct. More calls also mean more latency and more cost, and the coordinator can pick the wrong specialist or call too many of them.

My prototype has a root agent and a registry of STLC specialists. Routing depends on the request and the files or text attached to it. When I read the code for this report, that routing was still mostly the instructions plus whatever the model generated. The split helped me implement the thing. It is not a workflow engine, and nobody has verified it formally. The next hardening I would do is schemas between stages, some validation, and a place where a person has to approve.

2.4 Grounding, Traceability, and Human Review

Retrieval-Augmented Generation (RAG) is the usual answer when a model needs facts it should not invent. Lewis et al. retrieve documents and then generate from them [7]. In testing that would mean using approved requirements, policies, API specs, or old defects, instead of only the pasted prompt. Right now the prototype uses what the user attached, and whatever is still sitting in the in-memory session.

Even with retrieval, an evaluation has to tell two things apart: whether a case can be traced to a requirement, and whether the case is actually right. People still have to catch gaps and unsupported claims. If the output could affect a release, the evidence should stay on screen, uncertainty should be visible, and someone should have to accept it.

2.5 Evaluation of AI-Assisted Testing

If I were running the study now, I would want a baseline, inputs that look like real tickets, a frozen model and prompt, and scores that can be repeated. The measures I care about are requirement-to-test links, scenario coverage, human-rated accuracy, time on the same tasks, defect classification where labels exist, and end-to-end latency. They are not interchangeable. A longer list of cases is not better coverage. A faster first draft is not less work if review and correction take the time back.

NIST's AI Risk Management Framework organises this around governance, mapping, measurement, and management [8]. For me that meant: record the model and config versions, write down how a reviewer should score an artifact, keep suggestions separate from verified results, and treat privacy and operational risk as part of the evaluation, not something to add later.

Chapter 3

Methodology and Implementation

3.1 Development Method

The project was carried out in an iterative manner, starting with identifying the challenges faced by quality teams, followed by designing and developing a working prototype. The work involved planning the overall architecture, building the frontend and backend services, integrating the AI agents, and identifying how the completed system could be evaluated. This approach was suitable for the project, as the main outcome was a working application that brings together different activities of the Software Testing Life Cycle.

The initial proposal outlined a phased approach, beginning with public and open-source projects to explore the system's capabilities, with the possibility of using anonymised organisational data in a controlled setting at a later stage.

To understand the actual state of the system, the implementation was reviewed across its main components, including the frontend, backend, agent service, test cases, and deployment documentation. The description of implemented features in this report is based on the functionality available in the code and the workflows that can currently be accessed through the application.

3.2 Requirements and Actors

The person I had in mind is a quality engineer, a developer, or someone on a release call who wants structured testing help, not another open-ended chat. After login they provide requirements or stories, maybe code differences or a test-run log, and then they read the reply. A follow-up can reuse a session id. An administrator picks the model and turns Jira or GitHub on through environment settings.

Requirement	Prototype behavior
Context submission	Accepts requirements, user stories, code differences, and optional session context through the authenticated assistance API.
Quality assistance	Returns a generated response that can include prioritized cases, test data, automation notes, and release risks.
Specialized analysis	Routes suitable requests to registered STLC agents for test design, execution-log analysis, change impact, defects, summaries, and readiness.
User separation	Associates assistance requests with the authenticated user and persists request inputs and responses in PostgreSQL.
Engineering context	Offers Jira Cloud story retrieval and GitHub pull-request or branch code-difference retrieval when configured.
Operational configuration	Supports provider/model selection, containerized local startup, health checks, and production configuration checks.

Table 3.1: Selected functional requirements and corresponding implemented behavior.

3.3 Technology Stack

Table 3.2 lists the technologies used in the prototype and the role each one plays.

3.4 System Architecture

There are four pieces: the browser UI, the API, the database, and the agent process. The React and TypeScript frontend calls the FastAPI backend. After the user is authenticated, the backend stores the request, builds the agent message, and later stores the reply. A separate service runs the root agent and the specialists through Google ADK. PostgreSQL holds users, requests, and the integration records.

Figure 3.1 matches what is in the code repository [3]. Figure 3.2 describes the high-level

3. METHODOLOGY AND IMPLEMENTATION

Layer	Technology	Role
Frontend	React 19, TypeScript, Vite	User interface, component rendering, and build tooling.
Routing	React Router	Client-side navigation between login, workspace, settings, and guide pages.
Markdown rendering	react-markdown, remark-gfm	Renders the formatted agent response, including tables, in the browser.
HTTP client	Browser Fetch API	Calls from the browser to the backend REST API.
Backend API	FastAPI (Python), Uvicorn	REST endpoints, request validation, and routing.
Agent orchestration	Google Agent Development Kit (ADK)	Root agent, nine specialised agents, tool use, and session handling.
LLM access	Gemini, LiteLLM, Amazon Bedrock, Ollama	Configurable model providers used by the agents for text generation.
Database	PostgreSQL	Users, assistance requests, and Jira and GitHub connections.
ORM and migrations	SQLAlchemy (async), Alembic	Data access and database schema migrations.
Authentication	JWT (python-jose), bcrypt	Access tokens and password hashing; Microsoft Entra single sign-on is optional.
Encryption	Cryptography (Fernet)	Encrypts stored Jira and GitHub credentials.
External APIs	HTTPX	Asynchronous calls from the backend to the agent service, Jira Cloud, and GitHub.
Deployment	Docker, Docker Compose	Runs the frontend, backend, agent service, and database together.
Configuration	Environment variables (`.env`)	Provider selection, credentials, and production settings.

Table 3.2: Technology stack used in the prototype.

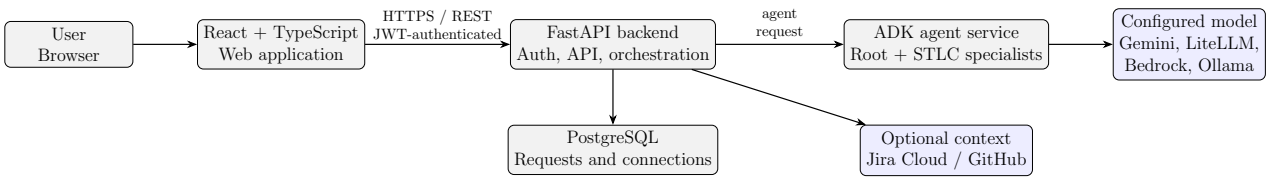


Figure 3.1: Logical application architecture. Optional external integrations and model providers are enabled by configuration.

design of the web application on AWS.

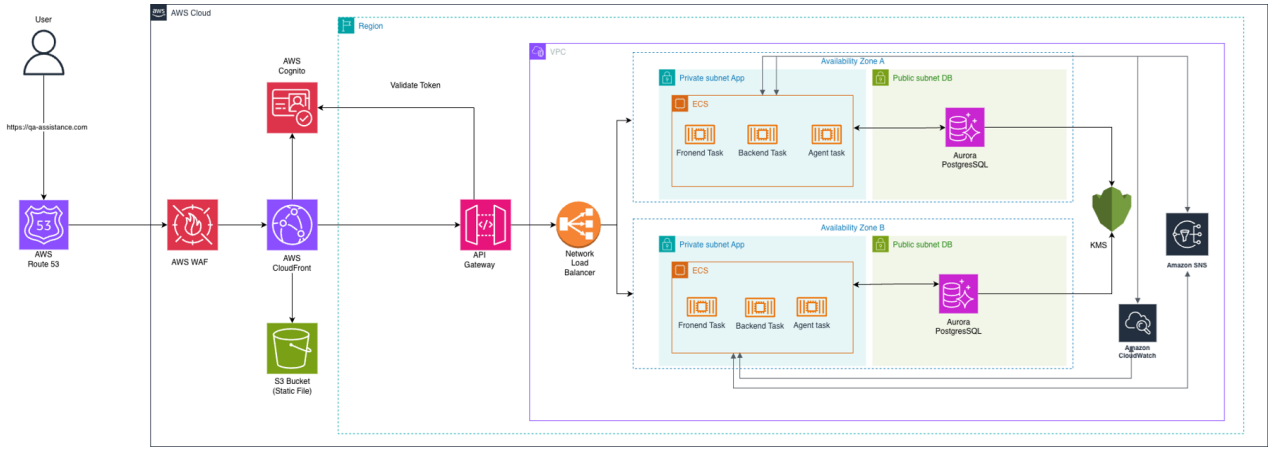


Figure 3.2: High-level design of the Quality Assistance application.

3.4.1 Frontend

The UI has login, a dashboard for requests, a settings page, and a short user guide. On the dashboard the user can type requirements and stories, paste a change, or start from the samples I added: smoke planning, an end-to-end example, change impact, execution and reporting, automation, and a follow-up. If Jira is connected, assigned issues can fill the story field. If GitHub is connected, a pull-request or compare code difference can be fetched. The reply is rendered in the browser. It can also be downloaded as Markdown or HTML.

3.4.2 Backend and Persistence

The backend is FastAPI, with asynchronous SQLAlchemy on PostgreSQL [9]. `/api/assist` needs a logged-in user. The request is stored with owner, source text, session id, status, timestamps, and later the response. Then the backend builds a structured message and calls the agent service. Both success and failure are written back. Errors go to the frontend on the normal error path.

Users, assistance requests, Jira connections, and GitHub repository connections are separate tables. That is enough to open an old submission. It is not enough if you expect the agent conversation to survive a restart. The ADK runner I used keeps session state in memory.

3.4.3 Agent Orchestration

The root agent is `quality_assistance_agent`. I built it with Google ADK, the configured model, logging callbacks, and the tools available at start-up [10]. On the supported cloud providers the root agent can pass work to the registered sub-agents. The backend prompt asks for prioritized cases, suggested data, automation notes, and release risks, using the requirements, stories, and code differences that came in.

ID	Specialist	Implemented responsibility
S01	Test case generation	Drafts structured, prioritized cases from requirements and stories, including positive, negative, and boundary scenarios.
S02	Test data generation	Suggests representative and edge-case test data.
S03	Test script automation	Suggests automation approaches and repeatable script outlines.
S04	Test execution	Interprets supplied or simulated test-run results and highlights failures; it does not itself launch the user's test suite.
S05	Self-healing test scripts	Recommends maintenance strategies when selectors or interfaces change; it does not directly edit repository files.
S06	Change impact analysis	Relates supplied code differences to potentially affected areas and regression scope.
S07	Bug report generation	Structures failure information into an actionable defect report.
S08	Test summary reporting	Summarizes test outcomes for stakeholders.
S09	Release readiness evaluation	Weighs supplied test and risk signals and provides a reasoned recommendation.

Table 3.3: Registered specialist agents and the scope described by their instructions.

Requirement and story text go to the test-case and test-data agents. Code differences go to change impact analysis. Execution logs go to execution, defect, and summary. Release questions go to readiness. The root agent also has a small requirement-summarization tool. All of this is prompt routing. There is no second engine underneath it that would force the same path every time.

3. METHODOLOGY AND IMPLEMENTATION

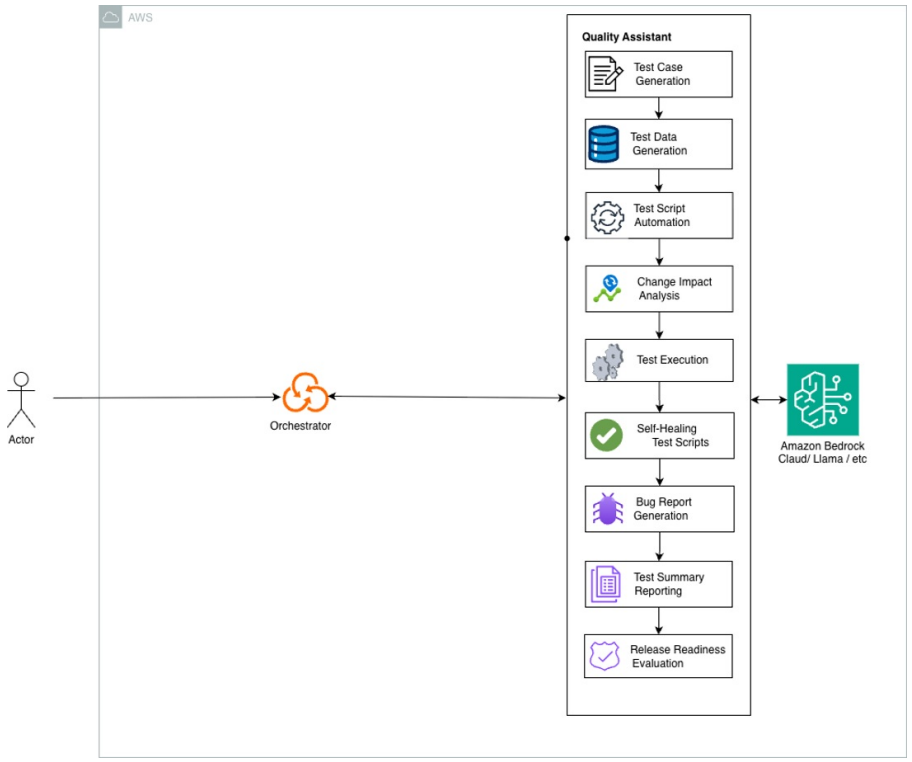


Figure 3.3: Illustrative agentic workflow showing orchestration among the root agent and specialist agents.

The steps for processing an assistance request are:

1. Validate the request and associate it with the authenticated user.
2. Persist the request inputs (requirements, optional stories, and code differences) with status `processing`.
3. Construct a message from the requirements, stories, and code differences.
4. Resolve or create an agent session for the user.
5. Invoke the root agent with the message and session.
6. Route to one or more specialists according to the request context.
7. Collect the generated response from the agent events.

8. If the agent call succeeds, persist the session identifier, response, and status `completed`; otherwise persist the failure status and return an explicit service error.

9. Return the request identifier, session, response, and status.

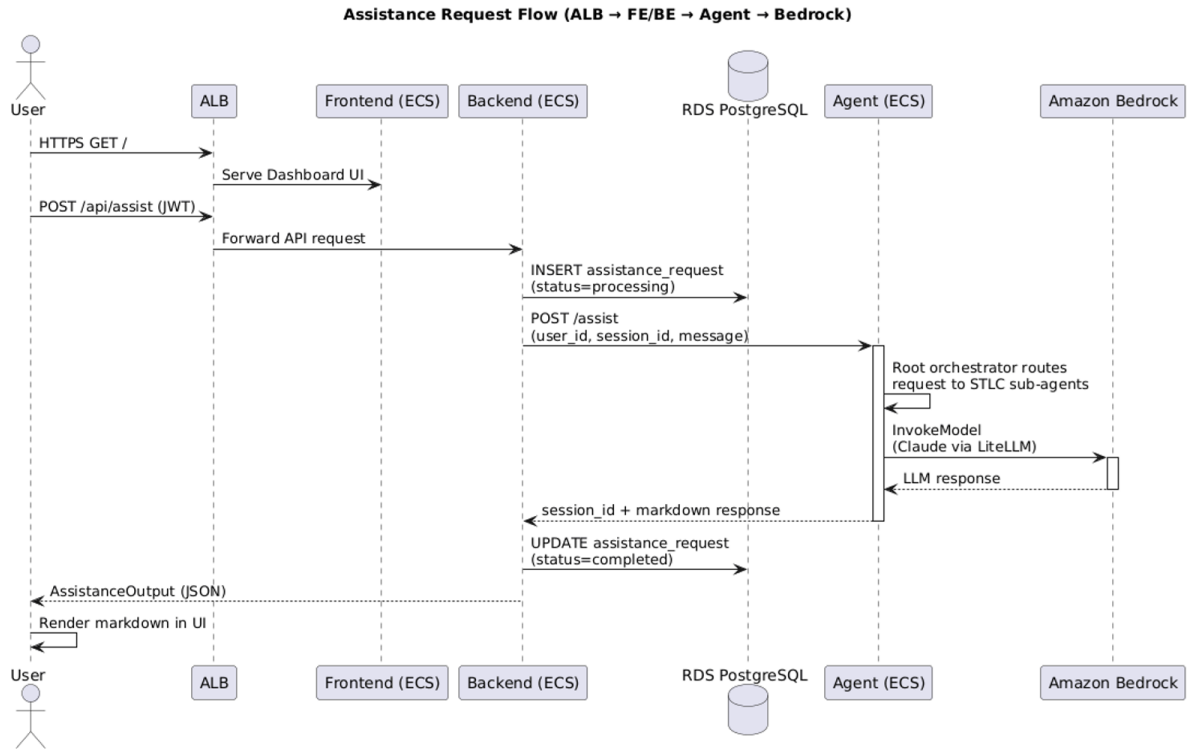


Figure 3.4: Illustrative request sequence through the application services.

3.4.4 External Integrations and Model Providers

Jira Cloud is OAuth 2.0, a short-lived state record, and Jira APIs to list assigned issues and fetch detail. GitHub settings store a repository and an encrypted personal access token. From there the backend can pull pull-request code differences or compare two references. Microsoft Entra SSO is optional. The agent side can use Gemini, LiteLLM, Amazon Bedrock, or Ollama if the credentials and models are there. I put those options in so the same code can run in more than one place. I did not benchmark every provider on the same tasks.

3.5 Security and Data Handling

Assistance records are scoped to the signed-in user. JWT tokens are issued after register or login. Passwords go through the project's password manager. Jira and GitHub credentials are stored encrypted. Jira OAuth state is a digest and it expires. In production the settings check expects real secrets, an encryption key, explicit HTTPS origins, database TLS, and non-default database credentials. The ECS note describes private app and database services, IAM roles, and managed secrets.

Requirements, stories, code differences, and test results are often confidential. The application stores them with the request and sends them to the configured model provider. Anyone who deploys this has to pick a provider and a retention policy the organisation will accept, limit who can see stored data, rotate credentials, and keep secrets out of the prompt. The checks I added reduce accidental leaks. They are not a privacy assessment, a threat model, or an internal security sign-off.

3.6 Deployment and Operations

For local use there is a Docker Compose file with PostgreSQL, frontend, backend, and agent. There are also scripts if you want the UI and Python services on the host, with the database local or in a container. The AWS ECS Fargate note is a pilot layout: separate containers, RDS PostgreSQL, an Application Load Balancer, and a chosen model. It is a checklist. It is not Terraform or CloudFormation, and I am not claiming that an AWS production stack has been brought up.

3.7 Quality Attributes

- **Modularity:** frontend, API, database, and agent run as separate processes. Specialists are registered on their own.
- **Configurability:** model, endpoints, integrations, and production flags come from the environment.

- **Traceability:** each user's inputs and replies are stored. I still do not have a formal requirement-to-case trail, and agent memory is not durable.
- **Availability:** health and readiness endpoints are there, and the container notes help. How well a real install holds up depends on the topology someone actually chooses.
- **Reliability:** the backend has error paths and settings checks. Generated test artifacts still need a person to read them, and they still need a proper study.

Chapter 4

Results and Discussions

4.1 Prototype Implementation Status

What I have at this point is a running full-stack prototype. It is more than a slide architecture. A user can register, sign in, send context through the authenticated API, and read a reply on the dashboard. Requests are stored. Nine specialists are registered, and the root agent picks among them from its routing instructions. GitHub code-difference retrieval, Jira Cloud OAuth and issue fetch, and optional Microsoft Entra login are in the same GitHub repository [3].

4.2 Existing Automated Test Assets

I wrote focused tests for Bedrock credentials, backend production settings, Jira Cloud behaviour, and Microsoft Entra SSO. Those files show that a few paths mattered enough to get checks. They do not tell you the suite is green today, and they do not cover the UI, the model, the database, and the agents end to end. If a later version of this report is going to claim test evidence, it should include the command, the environment, the commit, the test count, and the actual result of a run.

4.3 Demonstration Scenarios

The dashboard includes samples I used while building, so I could hit several agent routes without inventing a ticket every time:

Project area	**Status in inspected repository**	**Evidence or qualification**
Problem analysis and architecture	Implemented	Research framing is documented in the proposal and presentation; current services are separated into frontend, backend, and agent.
Core quality-assistance flow	Implemented	Authenticated request persistence, prompt construction, agent call, and response persistence are present.
Specialist STLC capabilities	Implemented	Nine task-focused agents are registered; orchestration is controlled by ADK and agent instructions.
Jira and GitHub context	Implemented, configuration-dependent	OAuth and issue retrieval are available for Jira; GitHub repository settings support code-difference retrieval.
Authentication and deployment controls	Implemented in part	Password/JWT authentication, optional Entra SSO, encrypted integration credentials, and production settings validation are present.
RAG over organizational test assets	Planned	No persistent requirements/test/defect retrieval index is evidenced in the inspected implementation.
Automated CI test execution and script mutation	Planned / out of scope of current behavior	Execution and healing agents analyze or recommend; they do not execute external suites or commit changes.
Comparative empirical evaluation	Pending	No measured baseline, benchmark results, or controlled organizational pilot results were supplied.

Table 4.1: Implementation status as of November 2026, distinguishing current behavior from planned work.

1. a password-reset requirement, for planning and summarising;
2. an e-commerce checkout with requirements, stories, and code differences;
3. an authentication change with large code differences, for regression impact;
4. a test-run log with pass, fail, and skipped cases, for execution analysis and reporting;
5. a Playwright selector change, for automation and self-healing notes; and
6. a follow-up that reuses the previous session id.

4.3.1 Interface Walkthrough and Illustrative Outputs

The screenshots are one ordinary path: open the workspace, check settings, type a requirement and a story, then read the reply. They show the screens and a sample output.

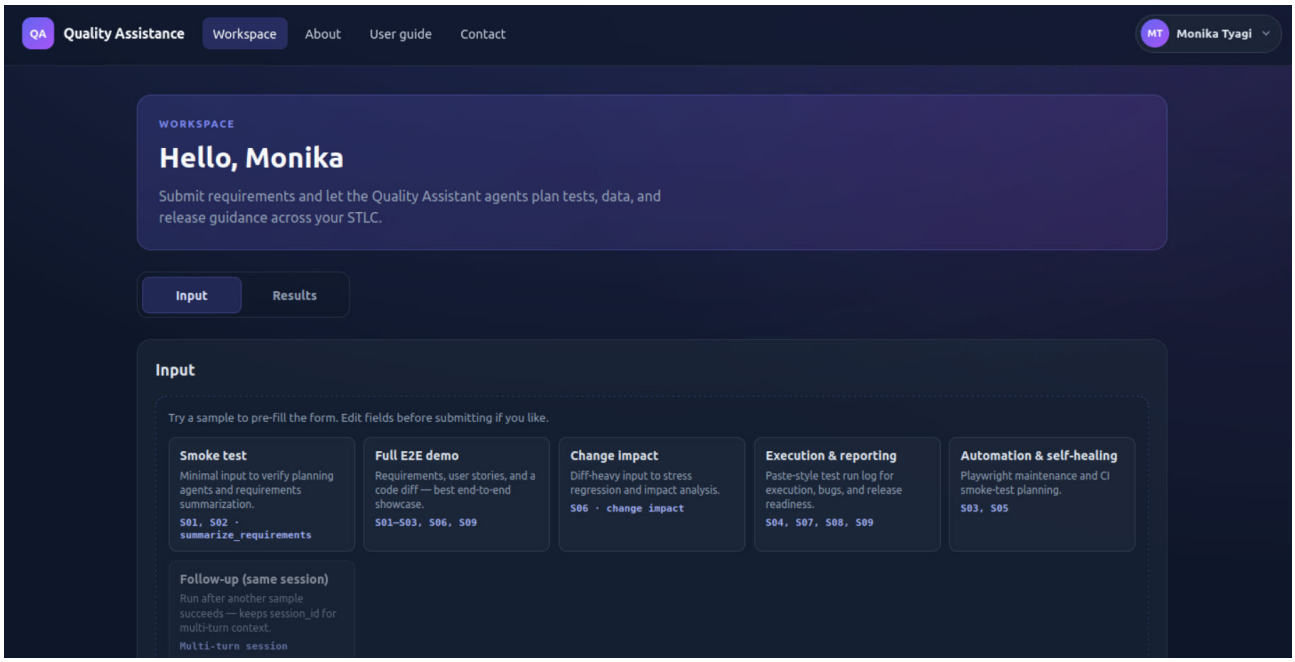


Figure 4.1: Workspace dashboard used to start a quality-assistance request.

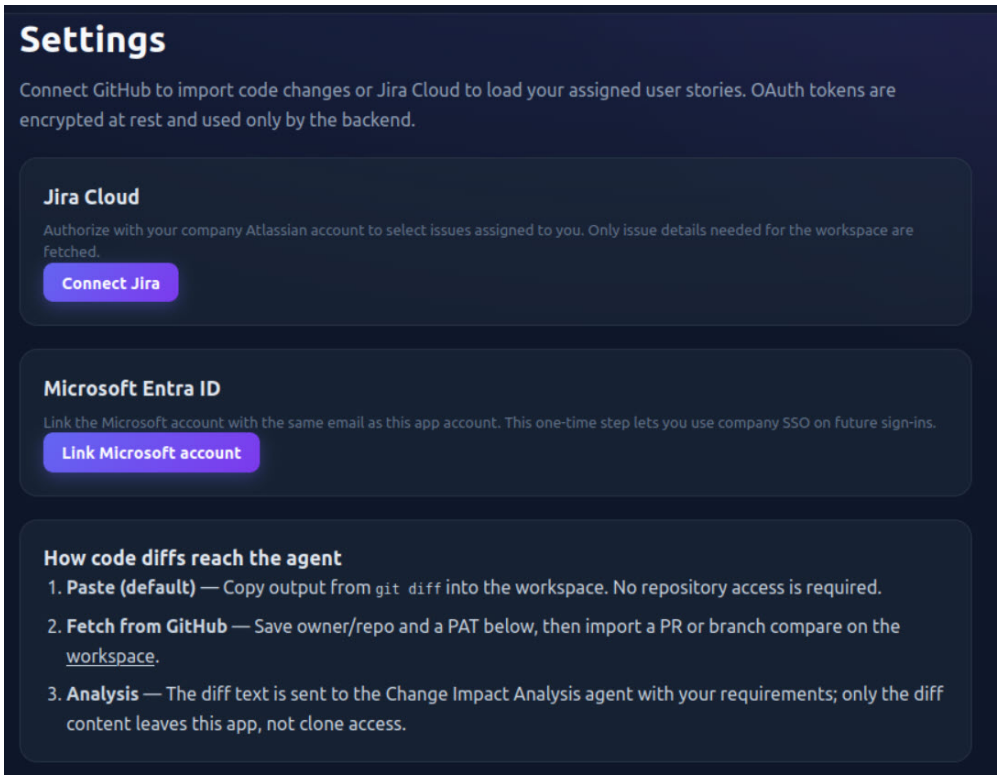


Figure 4.2: Settings page for configuring Jira, GitHub, and sign-in integrations. Availability depends on administrator configuration and valid credentials.

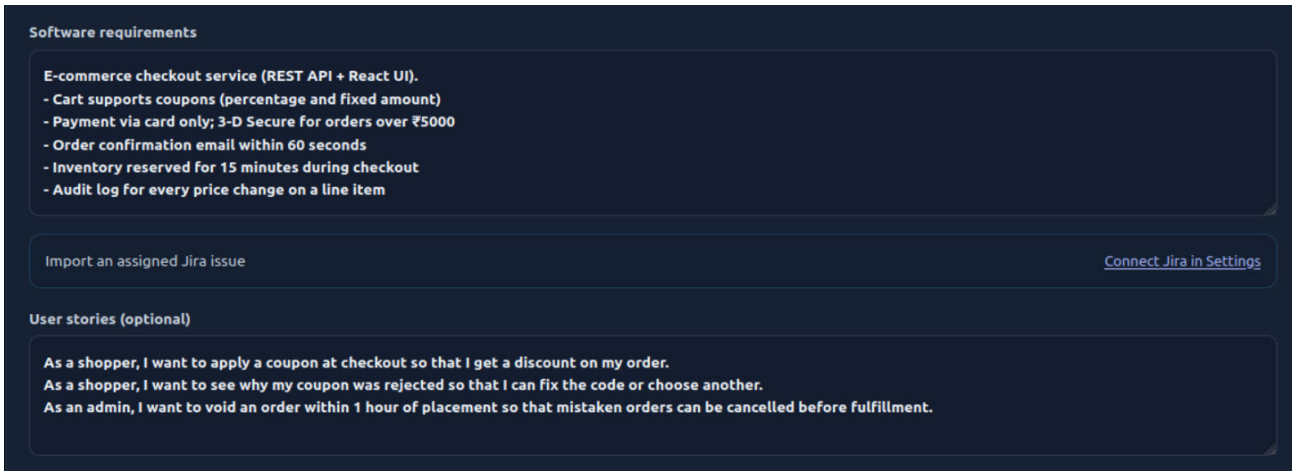


Figure 4.3: Example request input containing quality context and a user story before agent processing.

4. RESULTS AND DISCUSSIONS

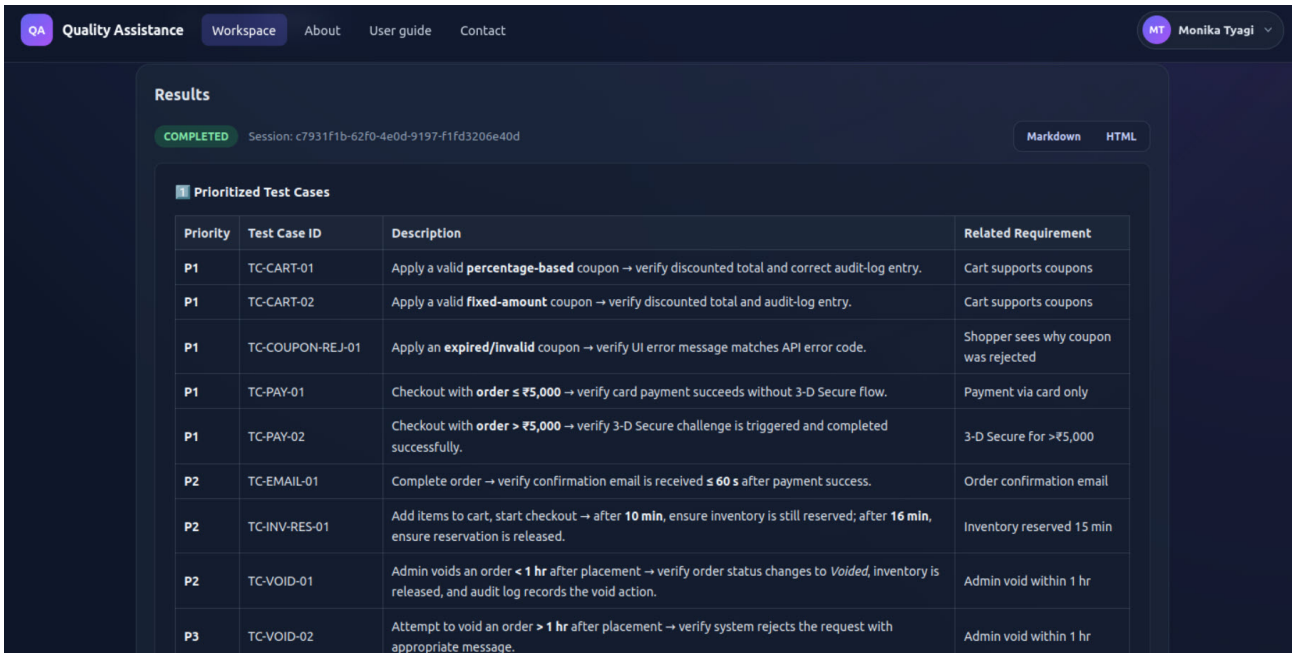


Figure 4.4: Example generated response with prioritized test-case recommendations.



Figure 4.5: Example suggested test data presented as part of the generated response.

4. RESULTS AND DISCUSSIONS

3 **Automation Notes** (for **test_script_automation** agent)

Area	Automation Approach	Script Outline / Tools
API Tests	Use **Postman/Newman** or **REST-Assured** for end-to-end flow (add to cart → apply coupon → checkout).	• Parameterize coupon codes, payment amounts • Validate response codes & JSON schema • Capture `orderId` for downstream checks
UI Tests	Selenium/WebDriver (or Cypress) for React checkout UI.	• Locate coupon input, submit and assert error/discount message • Mock 3-D Secure redirect via test gateway • Verify email-sent toast and order-status page
Email Verification	Use **MailHog** or **SMTP mock server** to capture outbound emails; assert receipt time.	• Start timer before order finalization • Poll mock server for matching `orderId` email • Assert timestamp ≤ 60 s
Inventory Timer	Background job simulation using **time-travel** in test environment (e.g., `clock` mock).	• After checkout, fast-forward 16 min and query inventory status via API • Verify reservation released
Audit Log Validation	Direct DB query or log-collector API (ELK).	• After price change, query `audit_log` table for recent entry • Assert fields: user, oldPrice, newPrice, timestamp
Admin Void	API call with admin token; combine with UI flow for admin portal.	• Submit void request within 1 hr, assert status `Voided` • Attempt after 1 hr, assert error message
CI/CD Integration	Add to **GitHub Actions** pipeline: 1 Unit → 2 API contract → 3 End-to-end UI (headless Chrome) → 4 Post-run report.	Use environment variables for secrets (payment gateway mock keys).
Self-Healing (future)	Tag UI elements with stable `data-testid` attributes; configure selector fallback strategy.	Include in **self_healing_test_scripts** recommendation later.

Figure 4.6: Example automation notes and test-script recommendations.

4️⃣ **Release Risks & Mitigations** (for **release_readiness_evaluation**)

Risk	Impact	Likelihood	Mitigation
Coupon mis-calculation (percentage vs fixed)	Wrong discount → revenue loss, poor UX	Medium	End-to-end price verification in every build; unit tests for discount engine
3-D Secure failure for >₹5,000 orders	Payment decline, cart abandonment	High (depends on external gateway)	Mock gateway in staging; monitor real-time success rate after release
Email latency >60 s	Customer confusion, support tickets	Medium	Deploy email service with SLA; add alert on queue depth; include performance test in CI
Inventory reservation timeout bug	Overselling, fulfillment issues	Low to Medium	Automated timer test; log monitoring for "reservation expired" events
Audit log omission	Compliance breach, debugging difficulty	Low	Enforce log write via middleware; unit test audit interceptor
Admin void window logic error	Orders stuck in pending state, financial reversal problems	Medium	Regression suite covering <1 hr and >1 hr cases; code review of time-window logic
Concurrency coupon application	Race condition causing double discount	Low	Stress test with parallel users; lock cart updates at DB level
React UI regression (broken data-testid)	Automation scripts break, manual testing overhead	Medium	Adopt `data-testid` convention; run UI regression smoke after each UI change

Overall Release Readiness – If all **P1** test cases pass in CI, email latency ≤ 45 s in load test, and audit-log coverage is 100 %, the release can be considered **green**. Any failure in P1 or a missed SLA (email, 3-D Secure) should halt promotion to production.

Figure 4.7: Example release-risk and readiness analysis based on the submitted context.

4.4 Evaluation Plan

When I do the study, I want three conditions on the same tasks: a person doing the work by hand, whatever point automation the team already has, and this assistant. The first round should stay on public requirements and open-source repositories or trackers, which is what the project plan already said. If we later use organisational material, it has to be anonymized, and it has to wait for the data-governance approval that organisation needs.

4.4.1 Dataset and Protocol

For each project I would pick a small, closed set of requirements, stories, code changes, and any test-run or defect records that already exist. Expected requirement-to-test links and expert-reviewed reference outputs should be written down before the assistant is run. Model, model version, prompt revision, and app config stay fixed for a comparison. Because the model does

not always give the same answer, I would repeat tasks where I can and keep both the raw output and the review decision. If it is practical, the person scoring the artifacts should not know which workflow produced them.

4.4.2 Measures

Measure	Operational definition	Interpretation
Requirement traceability	Percentage of in-scope requirements linked to at least one reviewer-accepted test case.	Tests whether generated artifacts preserve requirement coverage links.
Scenario coverage	Reviewer-rated coverage of agreed positive, negative, boundary, and risk conditions.	Measures breadth and relevance rather than the raw number of generated cases.
Artifact correctness	Proportion of sampled cases judged correct and actionable by independent reviewers.	Captures factual and procedural quality; report agreement between reviewers.
Effort	Active authoring and review time per requirement or change.	A reduction is meaningful only if review and correction time are included.
Defect analysis quality	Precision, recall, or F-score against adjudicated defect categories when labels exist.	Applies only where a suitable labeled reference set is available.
Response time and cost	End-to-end latency and model usage per completed task.	Quantifies operational overhead and helps assess scalability.
User acceptance	Structured ratings of usefulness, clarity, and trust from representative practitioners.	Complements objective measures but should not replace correctness review.

Table 4.2: Proposed evaluation measures; no values are reported because a controlled evaluation has not yet been completed.

4.4.3 Analysis and Validity

For numbers I would report sample size, the shape of the data, confidence intervals, and which tests I used. I would also look at error types, not only averages. On the qualitative side I would look for missing cases, claims with no support, unclear requirements, and places where the model and an expert disagree. Model and prompt versions have to be written down, otherwise a rerun later is hard to explain.

The threats I worry about are the usual ones. The dataset may not look like the work

people actually do. Reviewers will disagree. The model will not give the same answer twice. The provider may change the model under us. Examples may leak into prompts. And a long generated list may simply create more review. A small public-data run would be enough to talk about feasibility. It would not be enough to talk about enterprise workflows.

4.5 Results and Discussion

The evidence in this chapter is that the software exists and can be exercised. It takes quality context, stores a request, calls the orchestrated agent, and returns a response. The specialist list and the dashboard samples cover several STLC jobs. The Jira and GitHub code is a starting point for systems a team already has.

I do not have experiment results to put next to that. No benchmark set, no time-saved numbers, no coverage figures, no defect-detection rates, no user study. So I am not claiming the system improves quality or reduces effort. What this stage produced is the application and a way to test those claims later. A generated case can look complete and still be wrong, incomplete, or useless for finding defects.

4.6 Limitations

- If the user gives thin or wrong context, or if the chosen model is weak, the recommendations follow that.
- Routing between agents is still mostly the prompt. I did not add formal schemas or independent quality gates between stages.
- Session state is in memory. Restart the agent process and the conversation is gone.
- The system does not run external suites, edit scripts, or block a release.
- Retrieval over requirements, test assets, and old defects, and a Jenkins or similar CI hook, are still to be done.
- Tests cover a few settings and services. UI, model quality, load, and a full end-to-end path still need work.

- I have planning notes for production, not a signed-off deployment or an organisation-specific privacy review.

Chapter 5

Conclusions

This project aims to address the gaps between different software quality activities by developing an AI-assisted application that supports various stages of the Software Testing Life Cycle (STLC). The prototype brings together a React and TypeScript frontend, a FastAPI backend, a PostgreSQL database, and a separate agent service built using Google's Agent Development Kit (ADK). At the centre of the system is a root agent that understands incoming requests and directs them to nine specialised agents. Each agent focuses on a particular testing activity, including test-case design, test-data generation, automation suggestions, execution analysis, self-healing recommendations, change-impact analysis, defect reporting, test summaries, and release-readiness assessment.

Along with these capabilities, the application provides user authentication and stores submitted requests for future reference. It also integrates with Jira Cloud to access issue details and GitHub to retrieve code changes. Additional features include optional Microsoft Entra single sign-on and support for configuring different AI model providers.

The prototype shows how multiple testing-related activities can be brought together within a single application and supported through specialised AI agents. However, further evaluation is needed before drawing conclusions about its actual benefits. At this stage, there is no experimental evidence to establish whether the system improves test coverage, reduces the overall effort required by quality engineers, or helps identify more defects. The agents are intended to assist engineers by providing suggestions and organising relevant information, not

by replacing their judgement.

It is also important to understand the limitations of the current implementation. The execution analysis agent reviews available test results but does not run test suites on its own. Similarly, the self-healing agent suggests possible changes to automation scripts but does not apply those changes automatically. The release-readiness assessment provides supporting information for decision-making; the final decision to approve or reject a release remains with the responsible team.

5.1 Future Work

What I would do next, in order:

1. structured outputs and checks between specialist stages;
2. a place for a person to approve or edit test cases, and anything that could affect a release;
3. an evaluation of traceability, correctness, review effort, latency, and cost against a baseline written down in advance;
4. trials on public projects and, after approval, on anonymized organisational artifacts;
5. retrieval over approved requirements, test repositories, and historical defects;
6. a careful CI/CD hook for test execution, and session history that survives a restart and can be audited; and
7. load, reliability, accessibility, privacy, and security reviews before this is treated as a wider deployment.

I already know an LLM can write something that looks like a test case. The question I still cannot answer is whether this orchestrated assistant produces work that people can trace and use, often enough that the review it creates is still cheaper than doing the task without it. The prototype is the place to run that comparison. Screenshots of a working demo are not the comparison.

Appendix

5.A Representative Request and Response Flow

A request usually starts after login. The user enters some mix of requirements, stories, and code differences. The frontend posts that to the backend assistance route. The backend attaches the current user, saves the request as processing, and builds an agent message. The agent service reuses a session if one exists, otherwise it opens a new one, calls the root agent, and collects text from the events. The backend stores the reply and the final status and returns the request and session ids. For a follow-up, the same session id goes back with the next turn.

5.A Reproducibility Checklist for a Future Evaluation

- Write down the application revision, model provider, model id, prompt revision, and runtime config.
- Keep anonymized inputs and the reference cases used for scoring.
- Use the same tasks and the same review rubric for every baseline and for the assistant.
- Time generation separately from editing and review.
- Track what was accepted, edited, or rejected, and why.
- Report the scores and the failures. Do not drop unsafe or misleading outputs.
- Get approval before using organisational requirements, source code, or defect records.

5.A Project Materials

The proposal has the motivation, the questions, the phased evaluation, and the work plan. The implementation and deployment notes in the GitHub repository [3] are what I used for the as-built architecture and for the feature limits in this report.

References

[1] International Organization for Standardization. *ISO/IEC/IEEE 29119-1:2022: Software and systems engineering—Software testing—Part 1: General concepts*. ISO, 2022. 1, 5

[2] International Software Testing Qualifications Board. Certified tester foundation level syllabus, version 4.0. https://www.istqb.org/certifications/certified-tester-foundation-level, 2023. 1, 5

[3] Monika Tyagi. Quality assistance application: Project source code repository. https://github.com/monikatyagiisc/quality-assistance-app, 2026. Accessed November 2026. 3, 9, 17, 30

[4] Saleema Amershi, Andrew Begel, Christian Bird, Robert De Line, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice*, pages 291–300, 2019. doi: 10.1109/ICSE-SEIP.2019.00042. 6

[5] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. 6

[6] Lei Wang, Cheng Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wei Zhao, Zhe Wei, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024. 6

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. 7

[8] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, U.S. Department of Commerce, 2023. 7

[9] Sebastián Ramírez. Fastapi documentation. https://fastapi.tiangolo.com/. Accessed November 2026. 11

[10] Google. Agent development kit documentation. https://google.github.io/adk-docs/. Accessed November 2026. 12